

Part 1

1. Introduction

Hi, I'm J.T., and I'm a Senior Data Scientist with about 13 years of experience building production-grade machine learning, NLP, and GenAI solutions across healthcare, banking, and large-scale data platforms. In my most recent role at Reliant, I've been leading data science and AI work for a healthcare platform focused on patient risk prediction, real-time monitoring, and LLM-powered workflow automation, so this Principal Data Scientist role at U.H.G feels like a very strong match for me. A big part of my work has been designing end-to-end ML architecture, building and deploying scalable models, and making sure those systems are reliable, explainable, and safe in a regulated healthcare environment. I've worked hands-on with Python, PyTorch, TensorFlow, Spark, Kubernetes, AWS, and GCP, and over the last few years I've also built GenAI and RAG applications using tools like LangChain and LangGraph. What I think fits especially well here is that I'm very comfortable balancing architecture leadership with hands-on execution, whether that means building proof of concepts, productionizing models, or mentoring engineers through design reviews and collaboration. I've also partnered closely with clinicians, product teams, and compliance stakeholders, so I'm used to translating complex business problems into practical AI solutions that can actually be used in the real world. Overall, this role stands out to me because it brings together healthcare, advanced AI, and technical leadership, which is really where I've done my best work.

2. Tell me tell me more about the the business value. Describe the project a little bit more in detail. You mentioned that you handled patient charts? What kind of information did you extract and for what purpose?

Yeah, yeah... that's a good question.

So, um, the main business value of that project was really around reducing manual chart review effort and improving the quality and completeness of clinical data, which directly impacts care decisions and also reporting metrics. In healthcare, a lot of important information is still locked in unstructured patient charts—like scanned PDFs, handwritten notes, discharge summaries—so clinicians or reviewers usually spend a lot of time going through those manually.

What we did was build an end-to-end pipeline to extract structured information from those charts. Using OCR with Tesseract and models like LayoutLM, we were pulling out things like diagnoses, procedure codes, medications, lab results, vitals like blood pressure, and also timestamps—like when a screening or visit happened. We also extracted context, like whether a condition was active, controlled, or just mentioned historically, which is important for quality measures.

The purpose was twofold... first, to support clinical workflows—so instead of manually reading charts, clinicians could see a summarized and structured timeline of patient history. And second, from a business perspective, it helped improve reporting accuracy for quality programs. For example, identifying if a patient had their annual eye exam, or if their blood pressure was under control, things like that... which sounds very similar to what you described with Stars.

By doing this, we reduced manual review time quite a bit—like, we saw around 1,500+ hours saved per year—and also improved data completeness, which led to better capture of quality metrics. And since those metrics tie directly to reimbursement and compliance, it had a pretty clear financial and operational impact as well.

3. Hmm, you were doing some kind of starts program or CMS charts program? And for some other purpose, where did the extracted information go? Did you put them in some tables in a clinical app? I mean, how was the output consumed?

Yeah... yeah, got it... that's a good clarification.

So, um, the extracted information didn't just sit in a model output—we actually pushed it into structured data layers that could be consumed by downstream systems. Basically, after OCR and NLP extraction, we normalized everything into standardized schemas—like patient ID, condition codes, vitals, dates, and care events—and stored that in our data warehouse, mainly in Redshift for AWS clients and BigQuery for GCP.

From there, the data was used in a few different ways. First, we exposed it through APIs—like FastAPI services—so clinical applications could pull structured patient timelines. For example, a clinician could open a dashboard and see things like latest blood pressure readings, gaps in screenings, or missing documentation without reading the full chart.

Second, we also fed that data into analytics and reporting layers. We had dashboards in Tableau and Power BI where operations and quality teams could track things like compliance rates, care gaps, and trends across populations. That's where the business teams really got value—because they could monitor performance and take action.

And then... uh, one important part was feedback loops. We built a small UI where clinicians could review the extracted data, correct it if needed, and that feedback was captured and used for model retraining and evaluation.

So yeah, overall, the output was consumed across three layers—APIs for real-time apps, data warehouse for analytics/reporting, and human-in-the-loop tools for validation and continuous improvement.

4. What kind of tools do they sell?

Yeah... um, if I understand your question right, you're asking what kind of tools or solutions we provided as part of that platform, right?

So, we didn't really "sell" a single product like a packaged tool... it was more like a platform with multiple capabilities that healthcare clients could use depending on their needs.

The main one was the clinical intelligence dashboard... where clinicians and care teams could see structured patient timelines, risk scores, and care gaps—like missing screenings or uncontrolled conditions. That was probably the most visible piece.

Then we had the chart abstraction / chart review automation tool... this is where the OCR + NLP pipeline would process patient charts and automatically extract things like diagnoses, vitals, and procedures, so reviewers didn't have to read everything manually.

We also provided APIs and backend services... so clients could integrate the extracted data and predictions into their own systems, like care management platforms or internal reporting tools.

And, um... more recently, we added GenAI-based tools—like a clinical summarization assistant and a kind of chatbot interface—where users could ask questions about a patient and get grounded answers from the chart data.

So yeah... it was more like a suite of AI-powered tools—dashboard, automation, APIs, and GenAI assistants—all focused on improving clinical workflows and quality outcomes rather than a single standalone product.

5. And then, the whole team built that or you did build a project? What was it as a project you did?

Yeah... yeah, that's a good question.

So, um... it wasn't just like a one-off project—we actually built it more like a product, but internally and for enterprise clients, not like something you sell off-the-shelf publicly.

Basically, what we built was a clinical AI platform, and one of the core modules inside that was the chart review or chart abstraction capability. So instead of just doing analysis once, we designed it as a reusable system that could continuously process new patient charts, extract data, and feed it into workflows.

The chart review piece itself included a few things... like an automated extraction pipeline using OCR + NLP, a UI/dashboard where clinicians could review and validate the extracted data, and APIs so other systems could consume that structured output. So it became part of daily operations, not just a one-time deliverable.

And yeah... over time, we kept improving it—adding GenAI-based summarization, better extraction models, and feedback loops—so it really behaved more like a product that evolved, rather than a fixed project that ended.

So I'd say... it started like a project, but we intentionally turned it into a production-grade product/module that multiple teams and clients could use repeatedly.

6. Let us go into the implementation details. What was your process for charts?

Yeah, sure... so for the chart input process, the first step was getting documents from multiple source systems, because the charts were not coming in one clean format. Most of the time, the inputs were scanned PDFs, faxed documents, discharge summaries, lab reports, referral forms, and sometimes image-based files coming from EMR exports or document repositories.

So, um, we built an ingestion layer that accepted those files through batch drops, secure storage buckets, and API-based uploads depending on the client setup. Once a chart arrived, we assigned metadata first—like patient ID, encounter ID, source system, document type if available, and ingestion timestamp—because we needed strong traceability before any extraction started.

After that, we ran preprocessing on the files. That included document splitting, page classification, image cleanup, de-skewing, noise removal, resolution checks, and OCR readiness steps, because a lot of scanned charts were messy and low quality. Then we passed the cleaned pages through OCR, mainly Tesseract in the workflow I worked on, to get raw text plus layout coordinates.

From there, the pipeline moved into document understanding. We used layout-aware models like LayoutLM to combine the OCR text with spatial structure, so we could extract fields more accurately from forms, tables, and clinical notes. Then we ran downstream NLP logic to identify things like diagnoses, medications, vitals, procedures, dates, and care-gap related events.

One important thing was that we did not treat the whole chart as one document only. We usually broke it into sections or page types first, because the extraction logic for a vitals sheet is different from a physician note or lab report. That made the pipeline much more accurate and easier to maintain.

And then, uh, after extraction, we normalized the outputs into a structured schema and sent them for validation, storage, and downstream consumption. So overall, the input process was: secure intake, metadata tagging, preprocessing, OCR, layout-aware extraction, clinical normalization, and then delivery into APIs, warehouse tables, and reviewer tools.

7. Awesome. You said that your system was processing continuously. I was wondering if charts were coming through a queue or where they all available in bulk for some place?

Yeah... good question.

So, um, we actually supported both patterns, but in production most of the time it was queue-based, event-driven processing, not just bulk.

For the real-time or near real-time flow, charts were coming through a queue—like Kafka or Pub/Sub depending on the environment. So whenever a new document landed in storage, say an S3 bucket or GCS, it would trigger an event, and that event would push a message into the queue with metadata like file location, patient ID, and document type. Then our processing services would consume from that queue and start the pipeline.

This worked really well for continuous ingestion... like new charts coming in daily from providers or systems, so we could process them incrementally without waiting for large batches.

But at the same time... we also had bulk processing, especially in the beginning or for backfills. For example, when onboarding a new client or loading historical data, we would process thousands or millions of charts from a data lake in batch mode using Spark jobs.

So yeah... I'd say the system was hybrid:

Queue-based streaming for ongoing, real-time chart processing

Batch pipelines for historical loads, reprocessing, or model updates

And both flows were designed to use the same downstream extraction pipeline, so we didn't have two completely different systems.

8. Imagine. You have got the PDF document right now. And you used Tesseract as OCR tool. How exactly did you analyse for summarization and all of that? How did you extract exactly?

Yeah... yeah, exactly, that's a good way to frame it.

So after OCR gave us the text, the main goal was to convert that messy, unstructured text into structured clinical fields that we could actually use for analytics and quality tracking.

We grouped the extraction into a few categories:

First, patient and encounter info

Things like patient ID, date of birth, encounter date, provider name, facility... basically the metadata to link everything correctly.

Second, clinical conditions and diagnoses

We extracted diagnoses, ICD codes when available, and also normalized condition names. One important part was identifying context—like whether a condition is active, historical, or ruled out... because that really matters for quality metrics.

Third, vitals and measurements

This was big for quality use cases... like blood pressure (systolic/diastolic), BMI, HbA1c values, cholesterol levels, things like that. We also captured timestamps, because “when” it happened is just as important as “what”.

Fourth, procedures and screenings

We pulled things like eye exams, foot exams, lab tests, imaging, preventive screenings... basically anything that indicates whether a patient met a care requirement.

Fifth, medications

Medication names, dosage, frequency—mainly to understand treatment patterns and sometimes to support condition validation.

Sixth, temporal events / timeline building

We didn't just extract fields—we also organized them into a timeline, like when a diagnosis was recorded, when a test was done, when vitals were measured. That helped a lot for downstream reasoning.

And then... how we actually extracted them was a mix:

Layout-aware models (like LayoutLM) to understand sections and structure

NER-style extraction for entities like conditions, meds, labs

Rules and patterns for things like vitals (since formats are often consistent, like "BP: 120/80")

Medical normalization using dictionaries/ontologies

And finally, LLMs came in more at the end... like summarizing the patient state or helping interpret edge cases when the structured pipeline wasn't confident.

So yeah... the idea was not just "get text," but turn charts into structured, time-aware clinical data that could directly support things like quality metrics, gap detection, and reporting.

9. And what LLM tool did you use?

Yeah... so for LLMs, we didn't rely on just one model—we evaluated and used a few depending on the use case.

In production, we mainly used GPT-4 through Azure OpenAI, especially for summarization and reasoning tasks. It gave us the best balance of accuracy and consistency, which was important for clinical use.

We also tested and, in some cases, used models like Claude for longer context handling, and open-source models like LLaMA 2 and Mistral when we needed more control or cost optimization for certain clients.

And then, um... for domain-specific improvement, we did some fine-tuning and instruction tuning using LoRA, mainly on open-source models, to better align with clinical language and reduce hallucination risk.

So yeah... overall:

GPT-4 (Azure OpenAI) → primary for high-quality reasoning and summaries

Claude → for long documents and context-heavy tasks

LLaMA / Mistral → for cost-controlled or customizable deployments

And we selected the model based on accuracy, latency, cost, and compliance requirements for each use case.

10. Which AI tool do you use for development?

Yeah... yeah, got it... you're asking more about the actual tools I used day-to-day to build things.

So, um, for development, I mostly used Python as the main language, and I worked in Jupyter notebooks for experimentation—especially during data exploration, model training, and testing extraction logic.

For actual coding and building services, I used VS Code a lot... writing Python modules, APIs, and pipeline components there. Then we packaged everything using Docker, so it could run consistently across environments.

For ML and NLP work, I used libraries like PyTorch, TensorFlow, Hugging Face Transformers, and spaCy, depending on the task. For data processing, it was PySpark and Pandas.

We also used GitHub for version control, and for tracking experiments, we used tools like Weights & Biases and MLflow.

And then, um... for LLM workflows specifically, I used frameworks like LangChain and LangGraph to build RAG pipelines, prompt flows, and agent-style logic.

So yeah... if I summarize simply:

Jupyter → experimentation

VS Code → development

Python + ML libraries → modeling

Docker → packaging

GitHub + MLflow/W&B → tracking and versioning

LangChain/LangGraph → LLM workflows

That was basically my day-to-day development setup.

11. That is all my questions. Do you have any questions for me?

Yeah... yeah, I do have a couple questions.

First, um... I'm curious about the Chart IQ / chart chasing system—how mature is it right now? Like, is it already fully in production with stable pipelines, or are you still evolving parts of the extraction and modeling?

Second... how are you currently handling unstructured chart data? Are you mostly using rule-based and traditional NLP approaches today, or are you already integrating LLMs and RAG into the workflow?

Third... from a data science perspective, what would you say are the biggest challenges right now? Is it more around data quality, model accuracy, scaling the pipelines, or aligning with CMS/Stars requirements?

And then, um... I'm also interested in the team structure—how do data scientists typically collaborate with engineering and product? Is it more embedded, or more centralized?

And lastly... what would success look like in the first 3 to 6 months for someone in this role?

Part 2

0. Give me the senior level of introduction.

Answer:

Yeah, sure... so I'm a Senior AI/ML Engineer with around 13 years of experience, and in the last few years I've been mostly focused on MLOps and production AI systems, especially in healthcare. I've been working a lot on end-to-end machine learning lifecycle, like from data ingestion, feature engineering, model training, all the way to deployment, monitoring, and continuous improvement in production.

In my current role at R.H.P, I'm leading the development of a HIPAA-compliant virtual care platform, where I built scalable MLOps pipelines using tools like MLflow, GitHub Actions, Docker, Kubernetes, and Terraform. I worked closely with data scientists and clinical teams to deploy predictive models and also LLM-based solutions, including RAG pipelines using LangChain and FAISS, to support real-time clinical decision making. I also focused a lot on monitoring, drift detection, and performance optimization to make sure models stay reliable in production.

I've also worked across AWS, Azure, and GCP environments, so I'm comfortable building and managing cloud-based ML infrastructure, and integrating with EHR systems and healthcare data workflows. I think one of my strengths is combining MLOps, DevOps, and AI together to build systems that are not just working, but scalable, secure, and easy to maintain.

So yeah, overall, I would say my experience aligns very well with this role, especially around production deployment, CI/CD automation, and building reliable ML systems in healthcare.

1. Can you explain your experience in hyperparameter tuning and how you actually did it for your models?

Answer:

Sure! Hyperparameter tuning is one of the key steps in improving model performance. For most machine learning models, I usually start by defining the key hyperparameters to tune. For example, for tree-based models like XGBoost, I would tune parameters such as the learning rate, max depth, number of estimators, and subsample. I typically use a combination of grid search and random search methods. In the early stages, I prefer random search because it gives me a broader coverage of the hyperparameter space. After that, I focus the search on the region that shows promise, using grid search for finer adjustments. I always validate my models using cross-validation to ensure that the improvements I'm seeing are generalizable. For deep learning models like TensorFlow and PyTorch, I tune hyperparameters such as learning rate,

batch size, the number of layers, dropout rates, and optimizer choice. I use a manual, iterative approach where I change one or two parameters at a time, testing the model on validation data after each change. For NLP tasks, such as fine-tuning transformer models, I focus on learning rates, batch size, and gradient accumulation steps. Throughout the process, I track my experiments using tools like MLflow so I can manage the results efficiently and compare them easily.

2. Can you explain your experience with integrating SAP data?

Answer:

Certainly! I've worked extensively with SAP data, especially in supply chain contexts. The data we pulled from SAP could either be raw or already modeled in warehouses, and this presented different challenges. When we worked with raw SAP data, it was important to have close collaboration with business analysts to understand the relationships between entities like suppliers, purchase orders, and inventory. I would often build transformation layers using Python, Pandas, and SQL to clean and normalize that data before feeding it into AI models or analytics systems. In cases where SAP data was well-modeled, we could directly work with it in its prepared form, which allowed for faster analytics development. But even in those cases, I still had to ensure that the data was aligned with the business logic, so I worked closely with domain experts to make sure the data was meaningful. For data access, we utilized secure APIs or replicated SAP tables, always keeping in mind compliance and security requirements.

3. How do you handle situations when a project needs GenAI, but the solution is not well-suited for it?

Answer:

Great question! This happens more often than people realize. GenAI is a powerful tool, but it's not always the right fit for every problem. When I encounter a situation where people are pushing for GenAI but it doesn't fit the problem, I usually start by re-framing the question with the stakeholders. I ask them what they really want to solve and what their success criteria are. Then, I suggest a more traditional approach if necessary. For example, if the problem is more about structured prediction or classification, traditional machine learning or rule-based systems might work better. I also propose hybrid solutions sometimes, where we use GenAI for parts of the process (like summarization or question-answering) and combine it with traditional models for structured predictions. I always try to show them the value of the different tools and help them see that sometimes a traditional approach can be more reliable and efficient for their specific use case.

4. How do you handle data sensitivity when you don't have access to certain data or need to aggregate it?

Answer:

In situations with restricted data access, I've dealt with it by focusing on building solutions that respect security and compliance requirements. In many cases, we can't work directly with sensitive data, so I often rely on synthetic data, data aggregation, or anonymization techniques to simulate or mask the real data. For example, I've used data aggregation methods where sensitive fields are replaced with generalized statistics, such as averages or counts, so the model can still function without compromising security. Another technique I've used is creating secure, isolated environments where sensitive data is processed, and we only get the results or features needed for model training without exposing the raw data. Additionally, I've worked on building systems that stay within secure boundaries by using APIs to access only the data that's needed, ensuring that it's encrypted in transit and at rest. Data governance teams are critical in these situations, and I work closely with them to ensure that everything complies with the necessary regulations.

5. Can you describe your experience with front-end development, particularly in delivering tools for the end user?

Answer:

Of course! As a full-stack developer, I've always focused on delivering tools that are not just functional but also easy to use. When I work on the front end, I make sure that the tools I build align with the workflows and needs of the end users. For example, in a supply chain project, I built dashboards and search interfaces that allowed users to interact with complex AI models and data in a simple way. I used React to build dynamic, responsive interfaces, and I made sure to implement features like search, filtering, and data visualizations. My goal was always to ensure that users could get quick, actionable insights without needing to understand the technical complexity behind the system. I also put a strong emphasis on usability, so I work closely with product owners and analysts to test the UI and make sure that it's aligned with what the users need. Whether it's an AI assistant or a data visualization dashboard, I always focus on delivering a seamless, intuitive user experience.

6. Can you explain your experience in model evaluation and performance monitoring?

Answer:

Yes, model evaluation and performance monitoring are key to ensuring models are doing what they're intended to do. I typically use cross-validation during training to ensure models generalize well and don't overfit. Post-deployment, I monitor performance using metrics that make sense for the business context. For classification tasks, I look at metrics like accuracy, precision, recall, F1-score, and AUC-ROC. I also use confusion matrices to see where the model is getting things wrong. Additionally, I use monitoring tools like AWS CloudWatch or Prometheus to track metrics such as inference latency, throughput, and error rates. I've also set up systems for automatic model retraining when performance degrades or when there is data drift. I rely on tools like MLflow to track model versions and experiment metadata, so everything is reproducible and transparent.

7. How do you handle situations where a machine learning model isn't performing as expected in production?

Answer:

When a machine learning model isn't performing as expected in production, I follow a systematic approach. First, I check the logs to see if there's an issue with the input data or the inference pipeline itself. Sometimes, simple things like unexpected null values, incorrect feature scaling, or faulty input processing can degrade performance. If the model is still failing after that, I analyze the model's predictions and compare them with expected outputs to spot any trends in its failures. I then analyze whether there's been any data drift or if the underlying distribution of data has changed. To resolve this, I might perform a retraining cycle, either with updated data or by revisiting feature engineering. If the problem is more complex, I explore alternative algorithms, more robust validation, or change the way the model is serving predictions, such as using batch processing instead of real-time.

8. Can you walk us through a time when you had to troubleshoot an issue that involved multiple layers of a system (e.g., data, model, deployment, etc.)?

Answer:

Yes, this happens quite a bit. One example is when I was working on a model deployment in a healthcare environment where the model's predictions were accurate during training, but once deployed, it started showing erratic results. The issue was related to the data pipeline. The input data had been preprocessed correctly during training, but once in production, it was being fed with data that had small changes due to shifts in the operational environment. The first step was to inspect the data at each stage of the pipeline to ensure it was correctly transformed before feeding it into the model. After isolating the issue, I found that the data preprocessing stage was missing a critical transformation that had been applied during training. Once I fixed the pipeline and redeployed, the performance of the model improved. So, it was a combination of data integrity, preprocessing, and deployment-related issues.

9. What are your thoughts on scaling machine learning models in production?

Answer:

Scaling machine learning models in production can be complex but is crucial for real-world applications. First, it's important to separate model training from deployment so that scaling and versioning can be managed independently. In terms of scaling, I use infrastructure like AWS SageMaker or Kubernetes to handle autoscaling for real-time inference, adjusting the resources as demand increases. For batch processing, I use Amazon EMR or AWS Lambda to handle processing in parallel without overloading the system. When scaling, I also focus on optimizing latency and throughput. Using techniques like model quantization or running models on GPU instances can make inference more efficient. It's important to continuously monitor system health and performance, and I like to implement auto-scaling groups to adjust to changes in traffic demand.

10. How do you ensure that your machine learning models stay compliant with industry regulations, such as HIPAA or GDPR?

Answer:

Compliance is critical in industries like healthcare or finance. To ensure models stay compliant, I start by ensuring the data used for training is de-identified or anonymized as necessary, especially with healthcare data under HIPAA. In terms of GDPR, I ensure data processing follows consent management procedures and data minimization principles. I also make sure to implement strong data encryption both in transit and at rest. When deploying models, I use secure APIs and access controls, and I also ensure that there is an audit trail for any model decisions, so we can maintain transparency. Regular security reviews, access audits, and automated compliance checks are also part of my process.

11. How do you decide whether to use a pre-built model or to build a custom model from scratch?

Answer:

Deciding whether to use a pre-built model or create a custom one depends on several factors. If there's already a pre-trained model that can be fine-tuned to solve the problem efficiently and meets the business goals, then I prefer to use that model. For example, for NLP tasks, I often start with transformer-based models like BERT or GPT and fine-tune them on specific data. If the pre-built model does not provide the performance or flexibility needed for a specific domain, I would consider building a custom model. Custom models are necessary when the problem requires specialized features, or there is a need for specific data that the general pre-trained models might not have encountered.

12. Can you describe a time when you had to make a tradeoff between model performance and model interpretability?

Answer:

There's always a tradeoff between performance and interpretability, especially when you move from simpler models to more complex ones. For example, in a financial setting, we were building a credit risk model. Initially, we used a simple logistic regression model, which was highly interpretable and could provide clear explanations for the decisions it made, but its performance was lacking. We then switched to a gradient boosting model, which improved performance significantly but at the cost of interpretability. To balance the two, we incorporated SHAP values and LIME to provide post-hoc explanations of the more complex model. This way, we were able to maintain high performance while still providing a level of interpretability required by regulatory standards.

Part 3

Introduction:

So, um, I have around 13 years of experience, and over the last several years I've been heavily focused on data engineering and building scalable data platforms, even though my title has sometimes been in AI or data science.

In my current role at R.H.P., I'm basically working as a Senior Data Engineer in practice, where I design and maintain large-scale data pipelines using PySpark and SQL across both batch and streaming systems. I work with really large healthcare datasets—like 10M+ EMR records—and build reliable pipelines using tools like Kafka, AWS Glue, and BigQuery, making sure the data is clean, well-modeled, and ready for analytics and downstream systems.

I've also spent a lot of time on data modeling, performance tuning, and distributed processing, like optimizing Spark jobs, handling partitioning strategies, and improving pipeline latency and cost efficiency. On the cloud side, I've worked extensively with AWS and GCP, designing systems that are scalable, fault-tolerant, and production-ready.

Along with that, I take ownership of design reviews, code quality, and mentoring engineers, so it's not just implementation—I'm also helping drive best practices across the team.

So yeah... I'd say my experience aligns very closely with this role, especially around building and optimizing data pipelines, working with PySpark/SQL, and delivering scalable data solutions in cloud environments.

1. Interviewer: "Jason, what does R.H.P. do?"

Jason: "R.H.P. is a healthcare technology company focused on building data-driven solutions for providers and healthcare organizations. We have developed the 'Virtual Care Intelligence Platform,' which helps predict patient risk, supports real-time monitoring, and automates clinical workflows using AI. We work across multiple systems—EMRs, lab data, and unstructured documents—to create actionable insights and make healthcare more efficient."

2. Interviewer: "Jason, what is your experience with vector databases?"

Jason: "In my recent role, I worked with FAISS, Pinecone, and other vector databases for semantic search, particularly for unstructured healthcare data like clinical notes and scanned documents. We used embeddings to store document chunks and metadata, and for semantic retrieval, which allowed us to pull relevant patient records based on meaning, not just keywords. This improved clinicians' ability to search historical records and patient notes quickly and accurately."

3. Interviewer: "Can you give me a real-world example of how you used RAG workflows?"

Jason: "In healthcare, we built a RAG workflow for clinical decision support. For example, when a clinician asks for a report on a high-risk patient, the agent retrieves structured data like lab results and historical visits from Cloud SQL and BigQuery. Then, it fetches relevant unstructured documents like notes and discharge summaries using a vector database. It combines all this data and generates a grounded report, helping clinicians make better decisions faster."

4. Interviewer: "Can you explain what longitudinal data tracking means?"

Jason: "Sure, when I say longitudinal data tracking, I mean tracking a patient's data over time. In healthcare, we don't just look at one snapshot of data; we look at how things change over time—like changes in blood pressure, medication, or lab results. It's important because treatment decisions often depend on how a patient's condition evolves, not just what their data looks like today."

5. Interviewer: "How do you think healthcare data is different from other industries, such as finance or manufacturing?"

Jason: "Healthcare data is different because of its sensitivity and compliance requirements. You have PHI (Protected Health Information), so you need to be very strict about security, encryption, and auditability. The data is also more complex and inconsistent, often coming from various systems like EMRs, lab devices, and scanned documents. Plus, healthcare decisions have a higher impact—mistakes in data can affect patient care. Other industries like finance may have sensitive data, but they typically don't have the same level of regulation or data complexity."

6. Interviewer: "Can you describe your experience with the NVIDIA stack?"

Jason: "Yes, in my current role, we used NVIDIA A100 GPUs for training and inference. I worked with CUDA, cuDNN, and TensorRT to optimize model performance, especially for real-time use cases. We used **Triton Inference Server** for serving models efficiently, optimizing GPU utilization, and managing workloads across multiple models. I've also worked with **NCCL** for distributed training, scaling across multiple nodes. I'm comfortable with the NVIDIA ecosystem for optimizing deep learning models and improving inference speed."

7. Interviewer: "How do you handle schema evolution in healthcare data?"

Jason: "For schema evolution, we use tools like the **Confluent Schema Registry** to manage versioned schemas for both structured and unstructured data. In the case of healthcare, especially when dealing with HL7 or FHIR data, we focus on ensuring backward compatibility—so that newer versions of the schema don't break older systems. For unstructured data like clinical notes, we handle schema changes dynamically in our data pipeline, allowing new fields or missing data to be managed in a way that doesn't cause failures."

8. Interviewer: "Can you tell me more about how you would modernize a healthcare data lake setup for agent AI?"

Jason: "Sure, for a healthcare customer with a GCP setup, I would focus on creating a unified **retrieval layer** that combines data from Cloud SQL, BigQuery, GCS, and other sources. I would use **vector databases** for semantic search over unstructured content like clinical notes and documents. For structured data, I'd integrate it into an easily accessible API layer with tools like **Cloud SQL** and **BigQuery** for real-time queries. I'd also use a **knowledge graph** to help agents reason across relationships in data. This setup would allow an agent to seamlessly query and retrieve data from multiple sources, whether structured or unstructured, for tasks like generating reports or answering complex queries."

9. Interviewer: "How would you handle a client's preference for a specific technology when it's not the best fit for their project?"

Jason: "I usually start by understanding why the client prefers the technology and then explain the pros and cons of the technology based on their specific needs. I make sure to present facts—like how the alternative might be more efficient, cost-effective, or scalable. I also offer a POC or benchmarks to show how things would work in practice. I avoid being too rigid about my preference and try to keep the conversation solution-focused, always ensuring that the client understands their options clearly."

10. Interviewer: "Can you describe how you handle different types of healthcare data (unstructured, structured, analytical) in your data engineering work?"

Jason: "For healthcare data, I first categorize the data into structured (like patient records), unstructured (like notes, PDFs), and analytical (like quality metrics). I build **ETL pipelines** for structured data to load into Cloud SQL or BigQuery. For unstructured data, I use **OCR** and chunking, then store it in a **vector database** for semantic search. I integrate all these sources using a unified retrieval layer, ensuring agents can access the right data depending on their needs, whether it's for real-time queries or long-term analytics."